Plataforma de Reservas de Coworking – Plan de Proyecto

Información general

Campo	Detalle
Proyecto	Plataforma de Reservas de Coworking
Tipo	Evaluativo – Calificado por el docente
Stack	NestJS + Next.js + PostgreSQL + Docker + Prisma
Integrantes	Juan Camilo Bernal Gordillo y Laura Marcela Quintero Poveda
Repositorio	https://github.com/laura-marcela05/Plataforma-de-Reservas-de-Coworking.git

Plan de Releases

Release 1 – Segundo corte

Objetivo: Entregar la API REST completa con arquitectura en capas y el frontend con las vistas de CRUD para las entidades base.

Sprint	Período	Festivos	Alcance	Historias de Usuario
Sprint 1 — Infraestructura y Entidades Base	Mar 18 → Mar 29	Mar 23	Docker Compose, Prisma schema, migraciones, módulos CRUD de Usuario y Sede	HU-01, HU-02
Sprint 2 — Espacios y disponibilidad	Mar 30 → Abr 10	Abr 2, Abr 3	Módulos CRUD de Espacio y TipoEspacio. Lógica de consulta de disponibilidad por fecha y horario	HU-03, HU-04
Sprint 3 — Gestión de Reservas	Abr 13 → Abr 17	—	Módulos de creación y cancelación de reservas con lógica de negocio. Frontend: listados y formularios de entidades base	HU-05, HU-06

Cierre segundo corte: 17 de abril de 2026

Release 2 -Tercer corte

Objetivo: Integración completa frontend ↔ backend, flujos avanzados (reservar → cancelar → historial), administración de tarifas y reportes de ocupación. Sistema validado y funcionando con Docker.

Sprint	Período	Festivos	Alcance	Historias de Usuario
Sprint 4 — Historial, Tarifas y Navegación	Abr 20 → May 8	May 1	Historial de reservas del usuario, administración de tarifas, navegación y layout general	HU-07, HU-08
Sprint 5 — Reportes y Consolidación Final	May 11 → May 22	May 18	Reporte de ocupación, listado de reservas activas del día, pruebas finales y validación con Docker	HU-09, HU-10

Cierre tercer corte: 22 de mayo de 2026

Historias de Usuario

HU-01: Registro de Usuario (CU-01)

Como usuario, quiero registrarme en la plataforma seleccionando un tipo de membresía, para acceder a los servicios de reserva de coworking.

Criterios de aceptación.

1. Se puede registrar usuarios con nombre, apellidos, correo, contraseña, teléfono y tipo de membresía (básica, premium, corporativa)
2. El correo tiene que ser único, si se duplica el sistema retorna un error.
3. Se puede consultar la lista de todos los usuarios
4. Se puede consultar un usuario por su ID

5. Se puede editar la información de un usuario existente.
6. Se puede eliminar un usuario que no tenga reservas asociadas.

Tareas Técnicas.

- Backend: DTOs (CreateUsuarioDto, UpdateUsuarioDto), Repository con Prisma, Service con validación de unicidad de correo y eliminación condicionada, Controller CRUD
- Frontend: Listado de usuarios, formulario de registro/edición, vista de detalle del usuario

HU-02: Gestión de sedes (CU-2)

Como administrador, quiero registrar, consultar, editar y eliminar sedes del coworking, para mantener la información de cada sede con si dirección, horario y capacidad.

Criterions de Acceptation.

1. Se puede crear una sede con: nombre, dirección, horario de apertura, horario de cierra y capacidad total.
2. El nombre de la sede debe ser único.
3. Se puede consultar la lista de todas las sedes.
4. Se puede consultar una sede por ID.
5. Se puede editar y eliminar una sede que no tenga espacios asociadas.

Tareas Técnicas.

- Backend: DTOs, Repository, Service con validación de nombre único y restricción por espacios asociados, Controller CRUD
- Frontend: Listado de sedes, formulario de creación/edición, detalle de sede

HU-03: Configuración de espacios (CU-03)

Como administrador, quiero configura los espacios disponibles en cada sede, para definir qué tipo de espacios ofrece cada sede.

Criterios de aceptación.

1. Se puede crear un espacio con nombre, tipo (escritorio individual, sala de reuniones, sala de conferencias, auditorio), capacidad y sede a la que pertenece.
2. La sede referenciada debe existir, si no existe retornar error.
3. Se puede listar espacios con filtro por sede.
4. Se puede editar y eliminar un espacio por reservas activas.

Tareas Técnicas.

- Backend: DTOs, Repository con validación de FK (sede y tipo), Service con validación de capacidad y restricción por reservas activas, Controller CRUD
- Frontend: Listado de espacios con filtro por sede, formulario de creación/edición, detalle de espacio

HU-04: Consulta de disponibilidad (CU-04)

Como usuario, quiero consultar la disponibilidad de espacios por fecha y franja horaria, para saber que espacios están libres antes de reservar.

Criterios de aceptación.

1. Se pue consultar la disponibilidad filtrando la sede, fecha, hora de inicio y hora de fin.
2. El sistema retorna solo espacios sin reserva activas en ese horario.
3. La hora de fin debe ser posterior a la hora de inicio.
4. Si no hay espacios disponibles, el sistema retorna un mensaje informativo.

Tareas Técnicas.

- Backend: Servicio de disponibilidad, query filtrada por sede/fecha/horario, validación de rango horario, endpoint GET /disponibilidad
- Frontend: Formulario de consulta de disponibilidad y vista de resultados

HU-05: Creación de reserva (CU-05)

Como usuario, quiero crear una reserva seleccionando un espacio, fecha, horario, para asegurar el uso de un espacio en el momento que lo necesito.

Criterios de aceptación.

1. Se puede crear una reserva con usuario, espacio, fecha, hora de inicio y fin.
2. No se permite reservar un espacio con una reserva activa en el mismo horario.

3. La hora de fin debe ser posterior a la hora de inicio.
4. La reserva registra automáticamente la fecha de creación y el estado.

Tareas Técnicas.

- Backend: DTO con validación de FKs, Repository con verificación de colisiones, Service con reglas de horario y registro automático de fecha/estado, Controller POST
- Frontend: Formulario de reserva con selects dinámicos, manejo de error por choque de horarios y confirmación de creación

HU-06: Cancelación de reserva (CU-06)

Como usuario, quiero cancelar una reserva, para liberar el espacio si ya no lo necesito.

Criterios de aceptación.

1. Se puede cancelar una reserva su faltan al menos 2 horas para la hora de inicio.
2. Si no cumple el tiempo mínimo, el sistema retorna un error.
3. Al cancelar, el estado de la reserva cambia a cancelada.
4. No se puede cancelar una reserva ya cancelada o finalizada.

Tareas Técnicas.

- Backend: Método cancelar () en Service con validación de tiempo mínimo y estado, endpoint PATCH /reservas/:id/cancelar

- Frontend: Acción de cancelación desde el detalle de reserva y actualización del listado

HU-07: Historial de reservas del Usuario (CU-07)

Como usuario, quiero consultar el historial de mis reservas, para revisar mis reservas pasadas, activas o canceladas.

Criterios de aceptación.

1. Se puede consultar el historial de reservas de un usuario por su ID.
2. El historial muestra el espacio, sede, fecha, hora de inicio, hora de fin y estado.
3. Las reservas se orientan de más reciente a más antigua.
4. Si el usuario no tiene reservas, retorna un mensaje donde se indica que no tiene reservas.

Tareas Técnicas.

- Backend: Query con joins (sede, espacio) ordenada por fecha, endpoint GET /reservas/historial?usuarioId
- Frontend: Página de historial del usuario con tabla de reservas y mensajes informativos

HU-08: Administración de tarifas (CU-08)

Como administrador, quiero configurar la tarifa por tipo de espacio, tipo de membresía, para que el sistema calcule el costo de las reservas.

Criterios de aceptación.

1. Se puede crea una tarifa con tipo de espacio, tipo de membresía y precio por hora.
2. No se permite duplicar la combinación tipo de espacio + tipo de membresía
3. Se ´puede listar, editar y eliminar tarifas.

Tareas Técnicas.

- Backend: DTOs con validación de combinación única, Repository, Service, Controller CRUD
- Frontend: Listado de tarifas y formulario de creación/edición

HU-09: Reporte de ocupación (CU-09)

Como administrador, quiero generar un reporte de ocupación por sede y periodo, para analizar el uso de espacios.

Criterios de aceptación.

1. Se puede generar un reporte filtrado por sede, fecha de inicio y fecha de fin.
2. El reporte debe mostrar total de reservas y espacios más utilizados.

3. Si no hay reservas en el periodo el sistema debe retornar un mensaje indicando que no hay reservas.

Tareas Técnicas.

- Backend: Endpoint GET /reportes/ocupacion filtrado por sede y fecha, query agregada (total de reservas y espacios más usados), Service con manejo de ausencia de datos
- Frontend: Página de reportes con filtros y tabla de resultados

HU-10: Listado de reservas activas del día (CU-10)

Como administrador, quiero listar las reservas activas del día para una sede, para tener visibilidad de la ocupación en el tiempo real.

Criterios de aceptación.

1. Se puede consultar el listado filtrando la sede.
2. El listado muestra el nombre del usuario, espacio, estado, hora de inicio y fin.
3. Si no hay reservas el sistema retorna un mensaje informativo.

Tareas Técnicas.

- Backend: Endpoint GET /reservas/activas?fecha&sedes, query filtrada por estado y día actual, Service con manejo de lista vacía
- Frontend: Página con listado de reservas activas del día para la sede seleccionada

Definition of Done (DoD) Global

Cada Historia de Usuario se considera terminada cuando cumple todos los siguientes criterios:

Backend

- Endpoint(s) implementados con arquitectura en capas: Controller → Service → Repository.
- DTOs con validaciones (class-validator) para campos como fecha, horario, capacidad, unicidad y FKs.
- Validaciones de reglas de negocio aplicadas (disponibilidad, horarios, reservas activas, unicidad de tarifas).
- Manejo de errores con excepciones HTTP apropiadas (BadRequestException, ConflictException, NotFoundException).
- Respuestas con formato uniforme mediante interceptor global.
- Endpoints probados manualmente con Postman/Thunder Client y funcionando correctamente.

Frontend

- Página(s) implementada(s) con componentes reutilizables (tablas, formularios, selects dinámicos).
- Consumo del API a través de la capa de services/.

- Manejo de estados: carga (loading), éxito y error en operaciones como reservas y disponibilidad.
- Formularios con validación del lado del cliente (campos obligatorios, rangos de horas, fechas válidas).
- Diseño responsivo y consistencia visual en todas las vistas.

Infraestructura y código

- Código versionado en GitHub con commits descriptivos.
- El sistema completo (backend + base de datos + frontend) funciona correctamente con docker compose up.
- No hay errores de consola ni advertencias críticas en frontend o backend.
- Las migraciones de Prisma están aplicadas y el esquema es consistente con el modelo de datos del coworking.
- Variables de entorno configuradas correctamente (.env).

Entidades del Modelo de Datos (Prisma)

Usuario *1------N* **Reserva**

Sede *1------N* **Espacio**

TipoEspacio *1------N* **Espacio**

Espacio *1------N* **Reserva**

Membresia *1------N* **Usuario**

Tarifa *N------N* **TipoEspacio+Membresia**

Resumen de entidades.

Entidad	Campos principales
Usuario	id, nombre, apellido, correo (unique), contraseña, teléfono, membresiaId
Membresia	id, tipo (básica, premium, corporativa)
Sede	id, nombre (unique), dirección, horarioApertura, horarioCierra, capacidadTotal
TipoEspacio	id, nombre (escritorio individual, sala de reuniones, sala de conferencias, auditorio)
Espacio	id, nombre, capacidad, sedeId, tipoEspacioId
Tarifa	id, precioHora, tipoEspacioId, membresiaId (unique compound)
reserva	id, usuarioId, espacioId, fecha, horaInicio, horaFin, fechaCreacion, estado (activa, cancelada, finalizada)

Cronograma de Releases

SEGUNDO CORTE (RELEASE 1) — CIERRE: 17 ABR 2026

BACKEND + FRONTEND BASE

SPRINT 1	SPRINT 2	SPRINT 3
MAR 18 → MAR 29	MAR 30 → ABR 10	ABR 13 → ABR 17
DOCKER	ESPACIO	RESERVAS
PRISMA	TIPOESPACIO	CANCELACIÓN
USUARIO	DISPONIBILIDAD	FRONTEND LISTADOS
SEDE	TARIFAS	FORMULARIOS
MEMBRESÍA		

TERCER CORTE (RELEASE 2) — CIERRE: 22 MAY 2026

INTEGRACIÓN + REPORTES

SPRINT 4	SPRINT 5
ABR 20 → MAY 8	MAR 30 → ABR 10
FRONTEND RESERVAS	REPORTE OCUPACIÓN
HISTORIAL	RESERVAS ACTIVAS
ADMINISTRACIÓN TARIFAS	PRUEBAS FINALES
NAVEGACIÓN	DOCKER VALIDACIÓN

Estado del Proyecto

- [x] Plan de releases revisado y aprobado
- [x] Historias de Usuario revisadas y aprobadas
- [x] Criterios de Aceptación revisados y aprobados
- [x] Definition of Done revisado y aprobado

- [x] Modelo de datos revisado y aprobado
- [x] Resumen de entidades validado
- [x] Cronograma de sprints estructurado
- [x] Documentación completa y organizada
- [ ] Repositorio GitHub creado
- [ ] Código base del backend iniciado
- [ ] Código base del frontend iniciado
- [ ] Integración API–Frontend
- [ ] Pruebas finales de funcionalidades
- [ ] Despliegue con Docker